Pemrosesan Bahasa Alami

Deep Learning for NLP — From RNN to Transformers

Martin Manullang

Institut Teknologi Sumatera
mctm.web.id

2026

OUTLINE

Peta materi pertemuan ini

1. **Rekap & Jembatan dari Sesi Sebelumnya**
2. **Dasar-Dasar Deep Learning**
3. **Dari MLP ke Model Sekuensial**
4. **Attention Mechanism**
5. **Transfer Learning & Transformers**

Capaian Pembelajaran

Yang bisa kamu lakukan setelah sesi ini

1. Memahami mengapa *Deep Learning* lebih unggul dari ML tradisional untuk teks.
2. Mengenal arsitektur **RNN, LSTM, dan BiLSTM** beserta konsep *attention*.
3. Memahami cara kerja **Transformer** dan *fine-tuning* model pre-trained.
4. Mampu melihat perbandingan performa antar model dari kode yang sudah berjalan.

Fokus Kelas Ini

Penjelasan lebih banyak **menggunakan intuisi visual** daripada rumus matematika yang rumit.

Mengapa Deep Learning?

Keterbatasan yang membuat kita perlu naik level

Pada sesi-sesi sebelumnya kita sudah pakai **TF-IDF + SVM** / **PyCaret**.

Ternyata ada beberapa hal yang **tidak bisa** dilakukan pendekatan itu:

- **Tidak peduli urutan kata** — "saya *tidak* suka" vs "saya suka *tidak* ada masalah" dianggap mirip.
- **Tidak paham konteks** — kata "bank" di kalimat "tepi sungai" vs "menabung di bank" dianggap sama.
- **Dimensi fitur meledak** — setiap kata unik menjadi satu kolom; ribuan kata = ribuan kolom.

Mengapa Deep Learning?

Apa yang ditawarkan Deep Learning

Deep Learning menyelesaikan ketiga masalah tadi dengan cara:

- **Belajar representasi sendiri** — model secara otomatis menemukan pola penting dari data, tanpa kita perlu rekayasa fitur manual.
- **Memori urutan** — arsitektur RNN / LSTM "membaca" kata satu per satu sambil mengingat apa yang sudah dibaca.
- **Konteks global** — mekanisme *attention* memungkinkan model melihat seluruh kalimat sekaligus.

Analogi Sederhana

Kalau TF-IDF seperti **kamus kata** yang hanya menghitung frekuensi, maka DL seperti **orang yang membaca** — ia mengerti konteks dan urutan.

Disclaimer Dataset

Peringatan Konten Sensitif

Dataset yang digunakan pada sesi ini berisi teks seputar **kesehatan mental** — termasuk topik depresi, kecemasan, dan pikiran bunuh diri.

Dataset ini diambil dari sumber publik dan digunakan **semata-mata untuk tujuan edukasi** penelitian Pemrosesan Bahasa Alami.

Jika kamu merasa terganggu dengan konten ini, silakan sampaikan ke pengajar.

Rekap & Jembatan dari Sesi Sebelumnya

Pipeline Traditional ML

Cara kerja yang sudah kita kenal

Setiap langkah dikerjakan **secara manual** oleh programmer:

- **Fitur dibuat manual**: kita yang memilih cara merepresentasikan teks (TF-IDF, BoW).
- **Model hanya belajar dari fitur tersebut** — bukan dari teks aslinya.

Pipeline Deep Learning

Cara kerja yang akan kita pelajari hari ini

Model **belajar sendiri** cara terbaik merepresentasikan teks:

- **Embedding** menggantikan TF-IDF — representasi kata *dipelajari* oleh model.
- **Neural Network** sekaligus menjadi ekstraktor fitur *dan* klasifier.

Perbandingan Pipeline: ML vs. DL

Di mana letak perbedaan utamanya?

Traditional ML

- Fitur **dibuat manual** (TF-IDF)
- Tidak peduli **urutan kata**
- Interpretasi lebih **mudah**
- Butuh **data lebih sedikit**
- Training **cepat**

Deep Learning

- Fitur **dipelajari otomatis**
- Memahami **urutan & konteks**
- Model lebih **kompleks**
- Butuh **data lebih banyak**
- Training **lebih lama**

$\Rightarrow$ Tidak ada yang selalu lebih baik — pilihan tergantung data, waktu, dan kebutuhan.

Dari Fitur Manual ke Representasi Terpelajar

Lompatan konseptual terbesar dalam Deep Learning

TF-IDF (Fitur Manual)

Setiap kata menjadi satu kolom angka.

Kata	dok.1	dok.2
`sedih`	0.42	0.00
`cemas`	0.00	0.55
`senang`	0.10	0.33

Kata yang mirip maknanya **tidak** saling terkait.

Word Embedding (Terpelajar)

Setiap kata menjadi **vektor kontinu** berdimensi kecil.

Kata	dim-1	dim-2
`sedih`	-0.8	$+0.3$
`cemas`	-0.7	$+0.4$
`senang`	$+0.9$	-0.2

`sedih` dan `cemas` **berdekatan** di ruang vektor karena maknanya mirip.

Dataset: Dari Sesi 1 ke Sesi Ini

Mengapa kita ganti dataset?

Sesi 1 — Indonesian Chat

- Bahasa Indonesia
- 2 kelas: *toxic* vs *non-toxic*
- Klasifikasi sederhana
- Cocok untuk ML tradisional

Sesi Ini — Mental Health Sentiment

- Bahasa Inggris
- **7 kelas** emosi/kondisi mental
- Nuansa makna sangat halus
- Membutuhkan pemahaman **konteks**

Dataset dengan 7 kelas yang mirip maknanya menjadi **alasan kuat** mengapa kita perlu metode yang lebih canggih dari TF-IDF.

Sekilas Dataset: Mental Health Sentiment

7 Kelas Label yang Akan Kita Prediksi

#	Label	Deskripsi Singkat
1	`Normal`	Teks tanpa indikasi kondisi mental tertentu
2	`Depression`	Ekspresi rasa sedih, putus asa, tidak berdaya
3	`Suicidal`	Pikiran atau niatan menyakiti diri
4	`Anxiety`	Kekhawatiran berlebih, rasa takut, panik
5	`Bipolar`	Perubahan suasana hati ekstrem (senang↔sedih)
6	`Stress`	Tekanan hidup, beban pikiran
7	`Personality Disorder`	Pola pikir atau perilaku yang tidak biasa

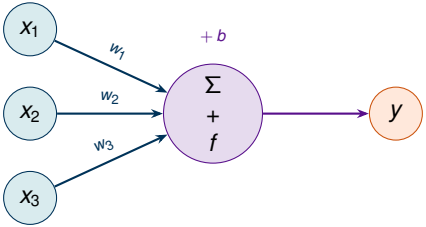
Tantangan: Banyak kelas yang sulit dibedakan — misalnya `Depression` vs `Suicidal`. Inilah yang membuat tugasnya menarik!

Dasar-Dasar Deep Learning

Neuron: Unit Terkecil Deep Learning

Meniru cara kerja sel saraf di otak

Bayangkan neuron seperti **saklar cerdas**: ia menerima beberapa sinyal, menimbang-nimbangnya, lalu memutuskan apakah harus "menyala" atau tidak.



- x_1, x_2, x_3 = **input**, w_1, w_2, w_3 = **weight** (bobot kepentingan)
- b = **bias** (pergeseran ambang), f = **fungsi aktivasi**
- Rumus: $y = f\left(w_1 x_1 + w_2 x_2 + w_3 x_3 + b\right)$

Weight & Bias: Parameter yang Dipelajari

Model belajar = model menyetel angka-angka ini

Weight (w)

- Menentukan **seberapa besar pengaruh** suatu input
- w besar $\rightarrow$ sangat berpengaruh
- w negatif $\rightarrow$ bersifat "menghambat"

Bias (b)

- Menggeser titik keputusan neuron
- Neuron bisa aktif meski semua input = 0
- Seperti b dalam $y = mx + b$

Inisialisasi Weight

Dimulai dari nilai acak kecil. Metode umum: **Xavier** atau **He initialization**. Nilai disesuaikan bertahap selama training.

Fungsi Aktivasi

Memberi kemampuan "berpikir non-linear" pada neuron

Tanpa fungsi aktivasi, seluruh jaringan hanya melakukan operasi linear.

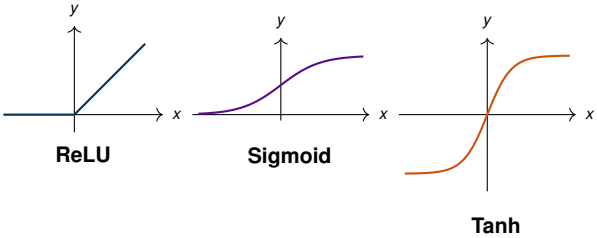
Fungsi	Rentang Output	Kapan digunakan
ReLU	$[0, \infty)$	Hidden layer — paling umum, komputasi cepat
Sigmoid	$(0, 1)$	Output layer klasifikasi *biner*
Tanh	$(-1, 1)$	Hidden layer RNN / LSTM — mengatasi batasan Sigmoid
Softmax	$(0, 1)$, jumlah = 1	Output layer klasifikasi *multikelas*

Analogi ReLU

ReLU ($\max(0, x)$) seperti lampu yang hanya menyala saat ada listrik positif — nilai negatif $\rightarrow$ tetap 0.

Grafik Fungsi Aktivasi

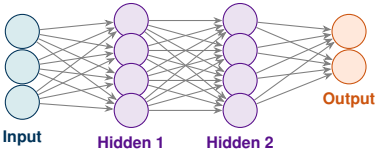
Visualisasi singkat bentuk masing-masing fungsi



Softmax tidak digambar di sini karena ia bekerja pada *vektor* (bukan satu angka) — mengubah sekumpulan skor menjadi distribusi probabilitas yang totalnya = 1.

Layer: Menyusun Neuron Menjadi Jaringan

Konsep *depth* dalam Deep Learning



- **Input layer**: menerima data mentah (teks yang sudah di-encode)
- **Hidden layer**: mengekstrak pola — makin banyak layer, makin *dalam*
- **Output layer**: menghasilkan prediksi (7 kelas $\rightarrow$ 7 neuron output)

Loss Function: Mengukur Seberapa Salah Model

Sinyal yang memberi tahu model "kamu masih keliru"

Loss function mengukur selisih prediksi model vs. jawaban benar. Makin kecil *loss*, makin bagus.

Cross-Entropy Loss — Klasifikasi Multikelas

$$\mathcal{L} = -\sum_{c=1}^{C} y_c \cdot \log(\hat{p}_c)$$

- $y_c = 1$ jika kelas c benar, 0 jika tidak
- $\hat{p}_c$ = probabilitas prediksi untuk kelas c (output Softmax)

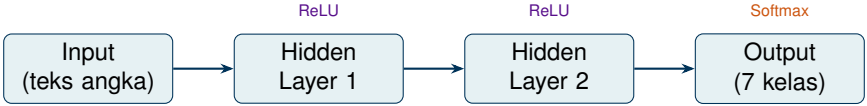
Contoh

Jawaban: `Anxiety`. Prediksi `Anxiety` 90% → loss kecil. Prediksi `Normal` 90% → loss besar.

Feedforward Network (MLP)

Neural network paling sederhana — data mengalir satu arah

MLP (*Multi-Layer Perceptron*) adalah jaringan saraf di mana data bergerak **hanya dari kiri ke kanan** — tidak ada putaran balik.



Kelebihan MLP:

- Sederhana dan cepat dilatih
- Cocok untuk data tabular

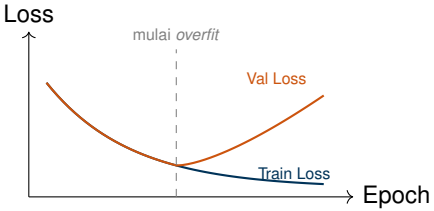
Kekurangan untuk Teks:

- Tidak mempertimbangkan **urutan kata**
- Inilah yang nanti diselesaikan RNN

Overfitting: Model Terlalu "Hafal"

Masalah klasik yang sering muncul saat melatih model

Overfitting terjadi ketika model terlalu fokus menghafal data latih, sehingga performa di data baru (data uji) justru buruk.



Saat *val loss* mulai naik sementara *train loss* terus turun, itulah tanda overfitting.

Regularisasi: Cara Mencegah Overfitting

Tiga teknik yang paling sering digunakan

Dropout

Saat training, sebagian neuron **dimatikan secara acak** di setiap iterasi.

Memaksa model tidak bergantung pada neuron tertentu saja.

Contoh: `Dropout(0.3)` $\rightarrow$ 30% neuron dimatikan tiap langkah.

L2 Regularization

Menambahkan **penalti** pada nilai weight yang terlalu besar ke dalam loss function.

Weight besar $\rightarrow$ model mengandalkan fitur tertentu secara berlebihan.

Disebut juga *weight decay*.

Early Stopping

Hentikan training **saat val loss mulai naik**, bukan saat mencapai epoch maksimum.

Parameter: `patience=3` $\rightarrow$ training berhenti jika val loss tidak membaik selama 3 epoch berturut-turut.

Dari MLP ke Model Sekuensial

Masalah MLP untuk Teks

Kenapa jaringan biasa tidak cukup?

Bayangkan kamu diminta menebak makna kalimat ini:

*"Saya **tidak** baik-baik saja"*

vs.

*"Saya baik-baik saja, **tidak** ada yang salah"*

MLP melihat kedua kalimat hanya sebagai **kumpulan kata** — kata "tidak" dan "baik" ada di keduanya. Ia **tidak peduli** urutan.

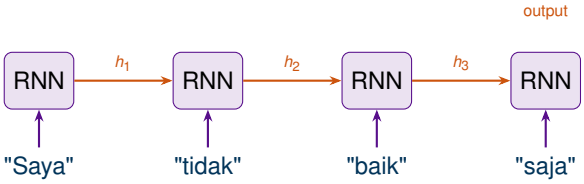
Inti Masalah

Teks adalah data **berurutan** (*sequential*). Makna sebuah kata bergantung pada kata sebelumnya.

RNN — Membaca Kata Satu per Satu

Seperti kamu membaca kalimat dari kiri ke kanan

RNN (*Recurrent Neural Network*) membaca teks **kata demi kata**, sambil menyimpan "ingatan" dari kata yang sudah dibaca.



h_t = **hidden state** — semacam "catatan" yang diteruskan dari satu kata ke kata berikutnya.

Hidden State: "Memori" RNN

Informasi yang dibawa dari kata ke kata

Setiap langkah, RNN menghitung hidden state baru:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

- h_{t-1} = ingatan dari kata sebelumnya
- x_t = kata saat ini (dalam bentuk vektor)
- Hasilnya h_t diteruskan ke langkah berikutnya

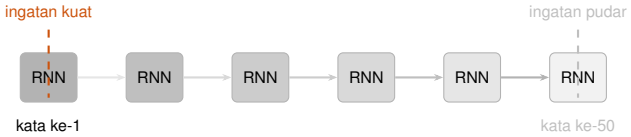
Analogi

Bayangkan kamu membaca novel sambil memegang **catatan kecil**. Setiap halaman, kamu update catatanmu. Di akhir bab, catatanmu merangkum semua yang terjadi.

Masalah RNN: "Lupa" Teks Panjang

Vanishing Gradient — musuh utama RNN

RNN bekerja bagus untuk kalimat pendek. Tapi untuk teks panjang, ada masalah besar:



Saat proses belajar (*backpropagation*), sinyal kesalahan yang dikirim ke belakang **semakin mengecil** tiap langkah — sampai akhirnya hampir nol.

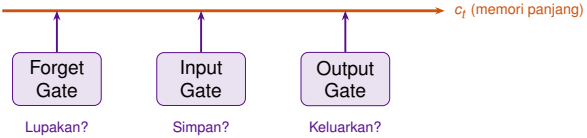
Akibatnya: RNN "lupa" kata-kata yang jauh di awal kalimat.

LSTM: Solusi untuk Ingatan Jangka Panjang

Long Short-Term Memory

LSTM menambahkan mekanisme **gerbang** (*gate*) — seperti filter yang memutuskan:

- Apa yang perlu **diingat** dari masa lalu
- Apa yang boleh **dilupakan**
- Apa yang perlu **dikeluarkan** sebagai output



Analogi

Seperti membaca buku dengan dua catatan: **catatan panjang** (ingatan penting) dan **catatan kerja** (konteks saat ini).

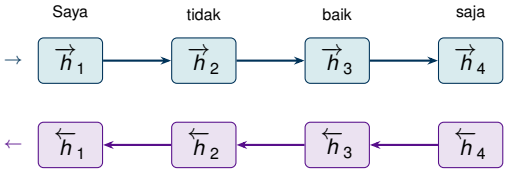
BiLSTM: Membaca Dua Arah

Bidirectional LSTM

Dalam bahasa, konteks datang dari **dua arah**. Contoh:

"Saya _____ sekali hari ini" — kata apa yang cocok?

Kita butuh tahu kata sebelum *dan* sesudah celah itu.



Output tiap kata = **gabungan** hidden state maju ($\overrightarrow{h}$) dan mundur ($\overleftarrow{h}$) — konteks dari dua sisi.

Word Embedding: Mengubah Kata Menjadi Angka

Representasi kata yang "memahami" makna

RNN tidak bisa langsung membaca kata — kita harus mengubahnya ke angka terlebih dahulu.

One-Hot Encoding (Cara Lama)

Kata	Vektor			
"kucing"	1	0	0	0
"anjing"	0	1	0	0
"mobil"	0	0	1	0

Masalah: "kucing" dan "anjing" **sama jauhnya** dengan "mobil" — model tidak tahu keduanya sama-sama hewan.

Embedding Layer (Cara DL)

Kata	dim-1	dim-2
"kucing"	0.82	−0.11
"anjing"	0.79	−0.08
"mobil"	−0.50	0.65

"kucing" dan "anjing" **berdekatan** — model belajar bahwa keduanya serupa.

Arsitektur BiLSTM di Workshop Ini

Gambaran model yang akan kita latih

Ringkasan Alur

Teks → indeks → embedding
→ BiLSTM 2 arah → prediksi kelas

Ukuran Model

≈7 juta parameter — lebih kecil dari BERT, cukup untuk 7 kelas.

Rangkuman: RNN vs LSTM vs BiLSTM

Evolusi arsitektur sekuensial

Aspek	**RNN**	**LSTM**	**BiLSTM**
Baca urutan	✓	✓	✓
Ingatan jangka panjang	×	✓	✓
Konteks mundur (kanan→kiri)	×	×	✓
Kompleksitas	Rendah	Sedang	Sedang–Tinggi

Kesimpulan

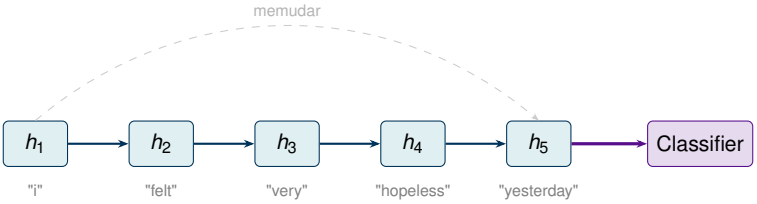
Untuk teks yang nuansanya halus (seperti dataset mental health kita), **BiLSTM** adalah pilihan yang jauh lebih baik dari RNN biasa.

Attention Mechanism

Masalah BiLSTM Murni

Last hidden state menanggung beban terlalu berat

Pada BiLSTM murni, setelah semua kata dibaca, **hanya satu vektor terakhir** yang diteruskan ke classifier.

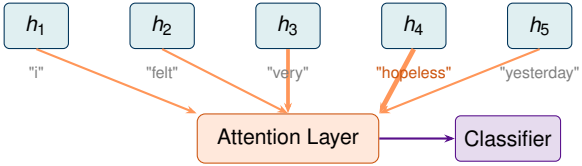


Kata *"hopeless"* di posisi ke-4 sudah **terkompresi** dan bercampur saat sampai ke posisi ke-5. Informasinya bisa memudar — terutama pada kalimat yang sangat panjang.

Solusi: Beri Model Kemampuan "Menyorot"

Ide dasar Attention Mechanism

Daripada hanya memakai *last hidden state*, bagaimana kalau model boleh **melihat semua hidden state** sekaligus dan memilih mana yang paling penting?



Panah yang lebih tebal = bobot attention yang lebih besar. Kata *"hopeless"* mendapat perhatian lebih dari model.

Cara Kerja Attention: Step by Step

Bahdanau Attention — langkah demi langkah

1. **BiLSTM menghasilkan hidden state** tiap kata: $h_1, h_2, \ldots, h_T$
2. **Hitung skor** — seberapa penting kata ini? $e_t = \tanh(W \cdot h_t + b)$
3. **Normalisasi** dengan Softmax (total = 1): $\alpha_t = \dfrac{\exp(e_t)}{\sum_{k=1}^{T} \exp(e_k)}$
4. **Buat context vector** — rata-rata tertimbang: $c = \sum_{t=1}^{T} \alpha_t \cdot h_t$

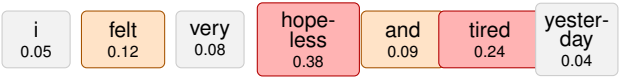
c inilah yang diteruskan ke classifier — merangkum seluruh kalimat dengan **penekanan lebih pada kata yang penting**.

Contoh Nilai α (Bobot Attention)

Apa yang "dilihat" model pada sebuah kalimat

Kalimat: *"i felt very hopeless and tired yesterday"*

Prediksi model: `Depression` dengan bobot attention:



Yang Model Fokuskan

hopeless ($\alpha = 0.38$) dan *tired* ($\alpha = 0.24$) — kata-kata yang paling menunjukkan kondisi depresi.

Manfaat Explainability

Kita bisa **menjelaskan** mengapa model memprediksi kelas tertentu — penting untuk aplikasi kesehatan.

BiLSTM Murni vs BiLSTM + Attention

Apa bedanya secara arsitektur?

BiLSTM Murni

1. Baca semua kata dua arah
2. Ambil **hidden state terakhir**
3. Masukkan ke FC + Softmax

Cepat, tapi info awal bisa memudar.

BiLSTM + Attention

1. Baca semua kata dua arah
2. Hitung **skor tiap hidden state**
3. Buat *context vector* c
4. Masukkan c ke FC + Softmax

Lebih akurat **dan** bisa dijelaskan.

Aspek	**BiLSTM**	**BiLSTM+Att.**
Teks panjang	Cukup baik	Lebih baik
Explainability	×	✓
Kecepatan train	Lebih cepat	Sedikit lambat
Parameter ekstra	—	W, b kecil

Transfer Learning & Transformers

Paradigma Transfer Learning

Belajar sekali, pakai berkali-kali

Bayangkan kamu sudah mahir membaca dan memahami teks bahasa Inggris. Ketika guru memberi soal analisis puisi, kamu **tidak belajar membaca dari nol** lagi — kemampuan itu sudah ada.

Transfer Learning di NLP bekerja persis seperti itu:

1. **Pre-train**: model dilatih pada data raksasa (miliaran kalimat) tanpa label khusus — model belajar *memahami bahasa*.
2. **Fine-tune**: model yang sudah "pintar" itu kita arahkan ke tugas spesifik kita — klasifikasi 7 kelas mental health — dengan data yang jauh lebih kecil.

Analogi

Seperti **dokter umum** yang melanjutkan spesialisasi. Ia tidak belajar anatomi dari awal — ilmu dasarnya sudah ada, tinggal dalami bidang tertentu.

Pre-train → Fine-tune: Dua Fase

Cara model besar menjadi berguna untuk tugas kita

Keunggulan

- Training kita jauh **lebih cepat**
- Butuh data berlabel **lebih sedikit**
- Performa biasanya **lebih baik**

Model Pre-trained Populer

BERT, DistilBERT, RoBERTa, GPT, T5 — semua bisa di-*fine-tune* untuk tugas kita.

Transformer: Model yang Mengubah Segalanya

Lahir dari paper *"Attention is All You Need"* (Vaswani et al., 2017)

RNN dan LSTM membaca kata **satu per satu** — proses berurutan dan lambat.

Transformer hadir dengan ide revolusioner:

"Bagaimana kalau semua kata dibaca *sekaligus*?"

Tiga komponen kunci Transformer:

Self-Attention

Setiap kata bisa "melihat" semua kata lain secara langsung — tidak perlu rantai.

Positional Encoding

Karena dibaca paralel, model perlu sinyal posisi agar tahu urutan kata.

Token [CLS]

Token spesial di awal kalimat yang merangkum maknanya — dipakai untuk klasifikasi.

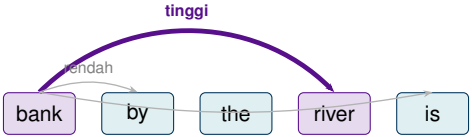
Dampak: Transformer menjadi fondasi BERT, GPT, T5, LLaMA, dan hampir semua model NLP terkuat saat ini.

Self-Attention: Setiap Kata Melihat Semua Kata

Inilah kekuatan utama Transformer

Kalimat: *"The **bank** by the **river** is flooded"*

Kata *bank* perlu "tahu" bahwa ia berada di dekat *river* agar model memilih makna **tepi sungai**, bukan **lembaga keuangan**.



Self-Attention menghitung skor keterkaitan menggunakan tiga vektor: **Q**uery, **K**ey, **V**alue.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Sederhananya: Q = "apa yang saya cari?", K = "apa yang tersedia?", V = "informasi yang dibagi".

Positional Encoding & Token [CLS]

Dua komponen pendukung Transformer

Positional Encoding

Transformer membaca semua kata *paralel* — ia tidak tahu mana kata ke-1 atau ke-5.

Solusi: tambahkan **sinyal posisi** ke setiap embedding kata.

"kucing makan tikus" $\neq$ "tikus makan kucing"

Tanpa positional encoding, kedua kalimat dianggap identik!

Token [CLS]

Token spesial yang disisipkan di **awal setiap kalimat**.

Setelah melalui semua layer Transformer, representasi **[CLS]** merangkum makna seluruh kalimat.

[CLS] inilah yang diteruskan ke classifier untuk menentukan label akhir.

[CLS] | i | felt | hope- | less

↓ ke classifier

DistilBERT: BERT yang Lebih Ramping

97% kemampuan BERT, 40% lebih kecil, 60% lebih cepat

BERT adalah model Transformer yang dilatih Google pada dataset raksasa — hasilnya sangat kuat untuk berbagai tugas NLP.

Masalahnya: BERT sangat besar dan lambat. **DistilBERT** hadir sebagai solusinya:

Aspek	**BERT-base**	**DistilBERT**
Jumlah parameter	110 juta	**67 juta** (-40%)
Kecepatan inference	$1\times$ (acuan)	**$1.6\times$ lebih cepat**
Akurasi (benchmark NLP)	100% (acuan)	**97%** dari BERT
Ukuran file	$\approx$440 MB	$\approx$**250 MB**

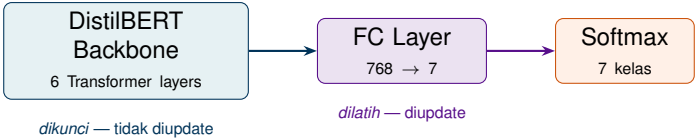
Knowledge Distillation

DistilBERT dilatih dengan cara "meniru cara berpikir" BERT, bukan hanya meniru jawaban akhirnya. Teknik ini disebut **knowledge distillation** — seperti murid belajar dari *proses penalaran* gurunya.

Fine-Tuning DistilBERT untuk 7 Kelas

Freeze backbone, latih classification head

Strategi yang dipakai: **freeze backbone** + tambahkan lapisan klasifikasi baru.



Freeze Backbone

Parameter DistilBERT **dikunci** — tidak berubah. Kita hanya melatih lapisan FC yang baru ditambahkan.

Keuntungan

- Training jauh **lebih cepat**
- Tidak perlu GPU terlalu kuat
- Mencegah *catastrophic forgetting*

Perbandingan Parameter: BiLSTM vs DistilBERT

Skala berbeda — implikasi berbeda

BiLSTM+Att. (≈7M)

- Ringan, cepat dilatih dari nol
- Cocok untuk resource terbatas
- Perlu cukup data berlabel

DistilBERT (≈67M)

- Backbone sudah "tahu bahasa"
- Butuh GPU, tapi fine-tune cepat
- Umumnya performa lebih tinggi

Perbandingan Final: BiLSTM vs DistilBERT

Kapan pilih yang mana?

Aspek	BiLSTM	BiLSTM+Att.	DistilBERT
Parameter	≈7M	≈7M	≈67M
Perlu GPU kuat	Tidak	Tidak	Disarankan
Pakai pre-trained	×	×	✓
Butuh data berlabel besar	Iya	Iya	Tidak harus
Explainability	×	✓(heatmap)	Terbatas
Akurasi pada 7 kelas	Baik	Lebih baik	Terbaik

Panduan Memilih

Resource terbatas / perlu explainability → BiLSTM + Attention.
Butuh performa terbaik / ada GPU → DistilBERT.